

that, let us look at a simpler question—what are the kinds of graphs that can not be bipartite? Also, let's see if we can identify some common properties that will allow us to decide whether a graph is bipartite or not.

Consider a simple line with two vertices. It is bipartite, since each vertex is in a different partition, and hence can be coloured with two colours. Next, consider a triangle with three vertices, A, B and C. If we colour A with one colour, and B with another, we do not have a colour left for the third vertex, since it is connected to both A and B by an edge. Hence, a triangle cannot be coloured with two colours, and therefore is not bipartite. A rectangle with four vertices, A, B, C and D, is colourable with two colours. A pentagon with five vertices can not be coloured with two colours.

Q As we go on increasing the number of vertices, what is the property that results in certain graphs not being *two-colourable*? Any graph that contains an odd cycle is not *two-colourable*, since an odd cycle is not *two-colourable*. Therefore, any graph that contains an odd cycle is not bipartite.

Hence, can there be graphs that do not contain an odd cycle, and are still not *two-colourable*? We leave it to you to prove that if a graph does not contain an odd cycle, it is bipartite. We now focus our attention on coming up with an algorithm to identify the presence of an odd cycle in a given graph.

To make it simpler, let's assume that the given graph is connected and that we have chosen an arbitrary starting vertex, s . Also, let's assume that we have two colours, black and white. Colour the source vertex s , white. Then colour any immediate neighbours of s , black. After that, process each neighbour of s , one vertex at a time, colouring each of their neighbours with white again. Repeat this until the whole graph is coloured. If the graph is *two-colourable*, all edges will have each of their vertices in different colours. Hence, the graph is bipartite. If an edge has both its vertices in the same colour (i.e., it is not colourable with two colours), it is not bipartite.

The algorithm described above is nothing but the breadth-first search (BFS) procedure. The only modification is to add a colouring step to the BFS. We colour all nodes that are at an odd number of edges from the root vertex with black, and all nodes that are at an even number of edges from the root, with white. Therefore, deciding whether a graph is bipartite or not can be done with a slight modification to the breadth-first search algorithm. I leave it to the reader to come up with the code.

This month's takeaway question

Q This month's takeaway question is about a well-known problem—the 'clustering of objects'. We are very often given a set of N objects, and have to partition them into k different groups or clusters, based on some property. For instance, we may be given a set of N images, and told to

cluster all similar images together, such that we have k clusters of images. All members of the same cluster have similar properties; members of two different clusters should be as dissimilar to each other as possible.

Typically, the similarity/dissimilarity is defined by a distance function between the objects. In graph notation, the objects are represented as points/vertices. We define the spacing to be the minimum distance between any pair of points present in different clusters. Since we want to group similar objects into the same cluster, and dissimilar objects in different clusters, our objective is to maximise the spacing, i.e., maximise the value of d , where d is the minimum distance between any pair of points (x, y) , where $x \in X$ and $y \in Y$; and X and Y are two different clusters. How would we do this?

The naïve approach is to list all possible k -clustering, and check which maximises the spacing. However, this approach is exponential in complexity, and hence impractical. The question for this month is to come up with a polynomial-time algorithm to perform the k -clustering. Though at first glance the problem seems difficult, I will give you a clue: recall our discussion of Minimum Spanning Trees. Can you apply MST algorithms and come up with a solution to the problem?

My 'must read book' for this month

This month's suggestion comes from our reader R Srikanth. He has recommended 'C Programming Language' by Kernighan and Ritchie. Srikanth says, "There are books on C which contain 500 pages and above, but teach you nothing. But here is a book that is just around 200 pages, yet covers all the concepts of C. More than that, this book teaches you how to write simple but efficient programs." I completely agree with Srikanth. This book is a must-read for every programmer.

If you have a favourite programming book that you think is a must-read for every programmer, please do send me a note with the book's name, and a short description on why you think it is useful. I will feature your choice in this column. It is sure to help many readers who want to improve their coding skills.

Also, do send me your answers to this month's questions. If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and wish you the very best! 

By: Sandya Mannarswamy

The author is a specialist in compiler optimisation, and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.

[If you are preparing for computer science programming interviews, you may find it useful to visit Sandya's programming interviews discussion group, 'Computer Science Interview Training [India]' on LinkedIn (www.linkedin.com), and her website <http://www.itinterviewtraining.com>]